NeuroCheck — Reporte técnico

Proyecto final de Compiladores · 2026-2
Estudiante: Adrian Alexander Benavides Caguasango
Integrantes adicionales: [Por completar, si aplica]
Institución: [Completar]
Fecha: Noviembre de 2026

Resumen

NeuroCheck es un analizador del lenguaje específico de dominio NeuroLang, destinado a describir redes neuronales densas. Reconoce tokens, verifica la sintaxis, construye una representación intermedia y valida que las dimensiones de las matrices de pesos sean compatibles con las capas. Incluye diagnósticos con ubicación y sugerencias, una interfaz web local en HTML, CSS y JavaScript servida por Python, y una herramienta de consola. El proyecto aplica técnicas de compiladores a un problema relacionado con IA, sin entrenar redes neuronales.

Palabras clave: compiladores, análisis léxico, análisis sintáctico, análisis semántico, lenguaje específico de dominio, redes neuronales.

1. Problema y motivación

En una capa densa, una matriz de pesos relaciona los valores de entrada con las neuronas de salida. En NeuroCheck se utiliza la convención `W[entradas,neuronas]`. Si una capa recibe cuatro valores y contiene dos neuronas, necesita una matriz `W[4,2]`. Una declaración `W[5,2]` contradice la estructura de la red. La discrepancia es un error de configuración que puede detectarse mediante análisis semántico, antes de entrenar un modelo.

El proyecto usa conceptos del curso para validar estas declaraciones de forma determinista. No pretende validar el contenido real de un entrenamiento ni comprobar que una red aprenda correctamente.

2. Objetivos

General: desarrollar un analizador léxico, sintáctico y semántico para configuraciones de redes densas escritas en NeuroLang.

Específicos: (a) diseñar la gramática; (b) reconocer tokens y construir árboles; (c) identificar errores y sugerir ajustes de dimensiones; (d) mostrar resultados en una interfaz; (e) validar con pruebas reproducibles.

3. Diseño del lenguaje

Ejemplo válido:

```
red MiRed {
  entrada 3;
  capa oculta: densa(4, pesos=[3,4]);
  capa salida: densa(2, pesos=[4,2]);
}
```

Reglas relevantes de la gramática (neurocheck/grammar.lark):

```
start: "red" NAME "{" statement* "}"
?statement: "entrada" INT ";" -> input_statement
        | "capa" NAME ":" "densa" "(" INT "," "pesos" "=" "[" INT "," INT "]" ")" ";" -> layer_statement
```

La gramática reconoce la forma de las instrucciones. Otras condiciones, como exigir al menos una capa, son reglas semánticas. NeuroLang no incluye asignaciones, condicionales ni ejecución de un programa neuronal.

4. Arquitectura y decisiones de diseño

Fase 1: análisis léxico. Lark reconoce palabras reservadas, identificadores, enteros y signos. Los tokens incluyen tipo, valor, línea y columna. Se usa una biblioteca generadora en lugar de implementar un scanner manual.

Fase 2: análisis sintáctico. El parser LALR de Lark comprueba la gramática. Ante una secuencia incorrecta, NeuroCheck informa E002. El árbol de análisis puede inspeccionarse desde la interfaz.

Fase 3: construcción del AST. Un Transformer convierte el árbol de Lark en un diccionario con el nombre de la red y las sentencias normalizadas (entrada, capa). Esta estructura evita analizar directamente el texto durante las validaciones.

Fase 4: análisis semántico. Se verifica la presencia de entrada y capas, los nombres duplicados, los límites de tamaños y la compatibilidad dimensional. Se construye una tabla de símbolos de capas. Un error E201 informa las dimensiones declaradas y las esperadas.

Fase 5: presentación. neurocheck/cli.py muestra resultados en terminal. app.py recibe solicitudes locales mediante /api/compile y llama a compile_source. El frontend en web/ presenta tokens, árbol, AST, símbolos y diagnósticos sin implementar reglas semánticas.

5. Regla matemática comprobada

Para dos capas consecutivas, si la capa anterior entrega n valores y la siguiente tiene m neuronas, NeuroCheck exige una matriz W de forma (n,m). Se verifica:

filas_declaradas == n y columnas_declaradas == m.

En el ejemplo, la primera capa requiere [3,4]; la segunda recibe los cuatro valores producidos por la primera y requiere [4,2]. Si el usuario declara [5,2], se emite E201 y se recomienda [4,2]. La herramienta no verifica los elementos numéricos de W.

6. Diagnósticos

Código	Situación
E001	Carácter desconocido para el lexer.
E002	Error de sintaxis según la gramática.
E101	Falta la declaración de entrada.
E102	Falta al menos una capa densa.
E103	Hay más de una declaración de entrada.
E104	Se repite un nombre de capa.
E105	Entrada fuera del rango educativo 1–64.
E106	Número de neuronas fuera del rango 1–64.
E201	La matriz de pesos tiene dimensiones incompatibles.

Los mensajes distinguen errores estructurales de errores de significado. Para entradas incorrectas no se construye un modelo validado, pero se muestran los tokens y la información disponible.

7. Implementación y archivos principales

- grammar.lark: gramática formal de NeuroLang.
- compiler.py: resultados, lexer, parser, AST y análisis semántico.
- cli.py: ejecución sin servidor web.
- app.py y web/: servidor HTTP local, editor, diagnósticos, tokens, árbol, tabla de símbolos y esquema de neuronas.
- tests/test_compiler.py: pruebas del compilador.
- neurocheck/visualization.py: generación independiente del diagrama SVG responsive.
- tests/test_visualization.py: pruebas de regresión visual estructural y del escalado del lienzo.
- examples/: seis archivos para demostración.

Se emplean Python 3.10+, Lark, HTML, CSS y JavaScript. pytest se usa para pruebas. La simulación de entrenamiento quedó fuera del alcance. En la revisión 1.2.0 se reorganizó la presentación: barra lateral de altura completa, paletas claro/oscuro coherentes, fondo integrado y visualización SVG adaptada a ambos temas.

8. Pruebas y resultados

El 11 de noviembre de 2026 se ejecutó python -m pytest -q sobre la versión 1.2.0: 30 pruebas aprobadas (18 del compilador, 6 de SVG y 6 de API y estructura web). Por CLI también se comprobó que una red válida informa dimensiones compatibles y que pesos=[5,2] produce E201 y propone [4,2].

Las pruebas cubren: tokens; árbol sintáctico y AST; tabla de símbolos; ejemplo válido; caracteres no permitidos; falta de punto y coma; incompatibilidad de matrices; nombres o entradas duplicados; declaraciones obligatorias; límites de tamaño; comentarios; serialización del resultado y varias capas consecutivas. Estas pruebas demuestran los casos implementados, pero no constituyen una demostración formal de ausencia total de errores.

Verificación de la interfaz: se revisó en Chromium con la misma página cargada directamente y respuestas equivalentes a las del servidor. Se verificaron temas, editor, tablas, selector de ejemplos y diagrama. Los endpoints del servidor se probaron por separado. La prueba final en el computador de los estudiantes sigue siendo necesaria.

9. Aporte original y extensiones

El aporte principal consiste en un lenguaje sencillo para declarar arquitecturas neuronales y en el algoritmo que detecta incompatibilidades entre capas, con sugerencias específicas. Como complemento, la aplicación permite inspeccionar el árbol sintáctico y visualizar la red validada. La suficiencia de estas aportaciones, especialmente en grupos de cuatro, debe confirmarse con el profesor.

10. Limitaciones y trabajo futuro

NeuroCheck analiza exclusivamente el DSL propuesto; no interpreta código Python general, no genera una red ejecutable, no consulta un modelo de IA y no entrena ni valida gradientes. La visualización limita a seis las neuronas dibujadas por capa por motivos de espacio, conserva el número real en las etiquetas y resume capas intermedias si son demasiadas. Como trabajo futuro se podrían comprobar valores reales de matrices, ampliar las capas admitidas o exportar la arquitectura a otro formato, pero no forman parte de esta entrega.

11. Contribuciones y trazabilidad

El listado final debe reflejar a cada integrante y el código realmente implementado, con respaldo en commits del repositorio. No se asignan tareas ni porcentajes ficticios antes de conocer y documentar el trabajo de los participantes. El archivo docs/06_CONTRIBUCIONES.md contiene una plantilla de registro.

12. Referencias

1. Documento del curso: *Proyecto Final Compiladores 2026-2* (guía suministrada por el docente).
2. Lark. *Lark: a parsing toolkit for Python*. https://lark-parser.readthedocs.io/
3. Python Software Foundation. *Python Documentation*. https://docs.python.org/3/
4. MDN Web Docs. HTML, CSS y JavaScript. https://developer.mozilla.org/
5. Asistencia de IA: ChatGPT (OpenAI), utilizada como apoyo para estructurar y revisar el código, las pruebas y la documentación. El grupo debe revisar y poder explicar cualquier artefacto incorporado.